Initiation au Développement : Algorithmique Fondamentale, Variables et Structures de Contrôle

Thème principal : Variables, Mémoire RAM, Structures Conditionnelles et Boucles Itératives

Synthèse exécutive : Ce cours pose les fondations indispensables de la programmation moderne et de la pensée algorithmique. Il résout le problème fondamental de la traduction d'un processus décisionnel complexe du monde réel en une suite d'instructions formelles, exécutables de manière déterministe par un ordinateur. On y apprend comment une machine réserve et manipule des emplacements en mémoire vive (RAM) via les variables, comment elle oriente le flux d'exécution selon des règles logiques (Si / Sinon), et comment elle automatise le traitement répétitif de données de masse grâce aux structures itératives (Pour / TantQue).

1. Mémorisation et Manipulation des Données : Les Variables en RAM

En informatique, une variable se distingue radicalement d'une variable mathématique. Il ne s'agit pas d'une inconnue dans une équation statique, mais d'un conteneur physique en mémoire vive (RAM) caractérisé par quatre éléments indissociables : un nom (identificateur usuel), un type (définissant la taille et le codage), une valeur actuelle et une adresse mémoire physique. Lors d'une affectation (ex: 'x ← 10'), l'ordinateur évalue d'abord l'expression située du côté droit, puis vient inscrire le résultat dans la case mémoire identifiée par le nom à gauche. L'ancien contenu de la case mémoire est alors immédiatement écrasé et perdu. Tenter de lire une variable non initialisée conduit à exploiter une valeur 'poubelle' résiduelle présente en RAM, source classique de dysfonctionnements.

- Variable : Emplacement mémoire étiqueté en RAM destiné à stocker une valeur modifiable au cours de l'exécution du programme.
- Typage : Spécification formelle du format de la donnée (Entier, Réel, Chaîne de caractères, Booléen) qui détermine la taille physique réservée en mémoire (octets) et les opérations autorisées.
- Affectation : Opération destructive d'écriture d'une valeur dans une variable, symbolisée par '←' ou '=', qui remplace sans retour possible l'ancienne valeur stockée.

Cas concret enseignant : *La métaphore de l'armoire d'archivage avec étiquettes : Imaginons la mémoire RAM comme une grande armoire munie de tiroirs. Déclarer une variable revient à coller une étiquette sur un tiroir (ex: 'solde_compte') et à dimensionner la taille du tiroir selon son type ('Entier'). Effectuer l'affectation 'solde_compte ← 150' consiste à ouvrir ce tiroir spécifique, à jeter l'ancien morceau de papier qui s'y trouvait, et à y déposer la nouvelle valeur 150.*

2. Prise de Décision : Les Structures Conditionnelles (Si / Sinon)

Par défaut, un programme s'exécute de façon séquentielle (ligne après ligne). Pour doter l'algorithme d'adaptabilité, on utilise les structures conditionnelles. L'unité centrale évalue un prédicat booléen : si la condition s'avère VRAIE, seul le bloc d'instructions sous 'ALORS' est exécuté. Si elle est FAUSSE, le bloc 'ALORS' est ignoré et le programme bascule dans le bloc 'SINON' (s'il existe). La maîtrise des tables de vérité est cruciale : dans une condition 'A ET B', les deux conditions doivent obligatoirement être vraies pour valider le bloc ; dans 'A OU B', la validation d'une seule des deux conditions suffit.

- Structure d'Alternative (Si / Sinon) : Aiguillage logique détournant le flux d'exécution séquentiel selon la valeur de vérité d'une condition.
- Expression Booléenne : Proposition logique n'ayant que deux états possibles : VRAI (True) ou FAUX (False).
- Opérateurs Logiques (ET, OU, NON) : Outils de combinaison permettant de composer des conditions multiples (Table de vérité).

Cas concret enseignant : *Le cas réel du distributeur automatique de billets : Avant de distribuer les coupures, l'automate évalue la condition complexe : SI (solde_client >= montant_demandé) ET (billets_disponibles >= montant_demandé). Si cette expression globale est VRAIE, le distributeur délivre les billets ; SINON, l'opération est refusée, un message d'erreur est affiché et la carte bancaire est restituée.*

3. Répétition et Automatisation : Les Structures Itératives (Boucles)

Les structures itératives exploitent la capacité du processeur à exécuter sans erreur des calculs répétés. La boucle POUR est une structure déterministe qui intègre l'initialisation, le test de sortie et l'incrémentation d'un compteur (ex: parcourir les éléments d'une liste de taille N). La boucle TANT QUE est indéterminée et dépend de l'évaluation d'un prédicat dynamique avant chaque tour. Risque majeur : si la variable intervenant dans la condition d'un TANT QUE n'est pas modifiée à l'intérieur du corps de la boucle, la condition reste éternellement VRAIE, provoquant une 'boucle infinie' qui gèle le programme.

- Boucle POUR (FOR) : Structure itérative bornée, privilégiée lorsque le nombre précis de répétitions est connu à l'avance.
- Boucle TANT QUE (WHILE) : Structure itérative non bornée, répétée tant qu'une condition logique demeure VRAIE.
- Incrémentation & Variable de Contrôle : Modification systématique d'un compteur pour garantir la sortie d'une boucle et éviter le blocage infini.

Cas concret enseignant : *La métaphore des consignes de l'entraîneur d'athlétisme : Si l'entraîneur ordonne 'Fais 10 tours de piste', il s'agit d'une boucle POUR (le nombre 10 est fixé à l'avance). S'il ordonne 'Cours tant qu'il ne pleut pas', c'est une boucle TANT QUE (la fin dépend d'un paramètre externe testé à chaque tour de piste).*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi l'instruction 'x = x + 1' est-elle valide en programmation alors qu'elle est impossible en mathématiques ?
Réponse du professeur : En mathématiques, '=' représente une égalité factuelle et statique. En informatique, '=' (ou '←') est un ordre d'affectation séquentiel et temporel. L'ordinateur calcule d'abord le membre de droite en prenant la valeur actuelle de 'x' augmentée de 1, puis il écrase l'ancienne valeur de 'x' en mémoire par ce nouveau résultat.

Question étudiant : Que se passe-t-il concrètement si j'oublie d'actualiser la variable de contrôle dans une boucle 'TantQue' ?
Réponse du professeur : La condition de boucle ne pouvant jamais basculer à FAUX, le programme rentre dans une 'boucle infinie'. Le processeur va exécuter le bloc d'instructions en continu sans jamais s'arrêter, provoquant le gel complet de l'application ou une saturation des ressources processeur.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un algorithme en pseudo-code qui demande à l'utilisateur de saisir un nombre entier positif N, puis calcule et affiche la somme de tous les entiers compris entre 1 et N inclus (1 + 2 + ... + N).

Résolution pas à pas :
Étape 1 : Déclaration des variables : N (Entier), i (Entier, compteur de boucle), somme (Entier, accumulateur). Étape 2 : Saisie utilisateur : Demander la valeur de N. Étape 3 : Initialisation : somme ← 0. Étape 4 : Boucle itérative : Pour i allant de 1 à N par pas de 1 Faire : somme ← somme + i FinPour Étape 5 : Affichage : Afficher le contenu de la variable 'somme'.

Résultat : Pour une saisie de N = 5, le déroulement mémoire est le suivant : i=1 -> somme=1 ; i=2 -> somme=3 ; i=3 -> somme=6 ; i=4 -> somme=10 ; i=5 -> somme=15. Le programme affiche correctement le résultat : 15.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'usage entre une boucle POUR et une boucle TANT QUE ?

R : La boucle POUR est utilisée lorsque le nombre exact de répétitions est connu avant le début de l'itération, tandis que la boucle TANT QUE est utilisée lorsque la fin de la boucle dépend d'une condition dynamique évaluée pendant l'exécution.

Q : Que vaut la variable A après l'exécution de la séquence : A ← 5 ; B ← 3 ; A ← A + B ; B ← A - B ?

R : A vaut 8. (Détail : A prend d'abord la valeur 5+3=8. L'instruction suivante modifie B [8-3=5], mais la valeur de A reste inchangée à 8).

Q : Pourquoi est-il impératif d'initialiser une variable (ex: somme ← 0) avant de l'utiliser dans un calcul de cumul ?

R : Parce que sans initialisation explicite, la case mémoire RAM associée contient des données aléatoires résiduelles ('valeurs poubelles'). Lire cette variable sans l'initialiser produit un résultat faux ou imprévisible.

Fiche académique générée et vérifiée par Kai pour Matéi • Hébergée sur http://88.96.47.180:8765